The Journal of Open Source Software

Smart-Geocubes: intelligent loading and caching of remote geospatial raster data

Tobias Hölzer [1,2], **Jonas Küpper** [1], **Ingmar Nitze** [1], and **Guido Grosse** [1,2]

1 Alfred Wegener Institute Helmholtz Centre for Polar and Marine Research, Telegrafenberg A45, 14473 Potsdam, Germany **2** Institute of Geosciences, University of Potsdam, Karl-Liebknecht-Str. 24-25, 14476 Potsdam, Germany

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 15 July 2026
Published: unpublished

Summary

Handling and analysing modern global geospatial datasets, for example satellite imagery or digital elevation models (DEMs), often requires data exploration and processing on subsets based on defined priorities or selected areas of interest. Furthermore, such data often comes from heterogeneous sources, is in various file formats, and has different projections, causing a substantial overhead in data management and preparation. Here we introduce `Smart-Geocubes`, an open-source library implemented in Python for seamless integration and efficient handling of global-to-local geospatial dataset processing tasks in state-of-the-art Earth Observation (EO) machine learning workflows and data pipelines. `Smart-Geocubes` provides an abstraction layer and utilities for accessing such data in an efficient and scalable manner by applying a local-first strategy in the form of `Zarr` datacubes, similar to a cache, for the purpose of organising and accessing a variety of geospatial data sources for machine learning applications.

Statement of need

Modern geospatial data workflows, such as training image segmentation models, combining datasets, or change detection analysis, often include an experimental phase where fast turnaround times and immediate access to data are crucial for rapid iteration of architecture and pipeline design. Local provision of data speeds up processing by avoiding network latency and bandwidth issues as opposed to on-demand cloud data. However, contemporary geospatial datasets are often too large for local storage; for example, `AlphaEarth embeddings` require approximately 500 TB, while the `Sentinel-2` archive is reported to exceed 40 PB and is still growing (Brown et al., 2025; National Oceanography Centre, 2026). Further, datasets are distributed across heterogeneous platforms (Google Earth Engine, Planetary Computer, STAC catalogues with S3 backends, etc.) with different access APIs and storage formats (`Zarr`, `GeoTIFF`, `COG`, `NetCDF`) (Gorelick et al., 2017). Each of those requires specific data discovery and transfer procedures as well as specialised conversion procedures and data loaders according to the end users' needs and the local storage requirements.

Users must therefore download spatial subsets (or "regions") of remote data while retaining the ability to incrementally add new regions without restructuring workflows. While pre-downloading regions of interest is a common and battle-tested method, `Smart-Geocubes` is designed to ease, improve, and unify this approach for geospatial researchers requiring local-first workflows with cloud-scale data. Thus, the requirements around which `Smart-Geocubes` was designed are (1) local-first data access with (2) procedural downloads of remote data from (3) various data-providers and (4) a low compute resource footprint.

State of the field

The tools closest in functionality to Smart-Geocubes are VirtualiZarr, xee, stackstac, odc-stac and cubo (Google LLC, n.d.; Joseph, n.d.; Montero et al., 2024; Nicholas et al., 2024; Open Data Cube, n.d.). xee, stackstac, odc-stac and cubo are very similar in functionality, as they let users create datacubes on demand based on remote archival data from STAC or Google Earth Engine. Data is never persisted locally, as they download the data into Xarray datasets, thus working completely in memory. VirtualiZarr does the same but with virtual Zarr datacubes and by storing the metadata locally while leaving the original data untouched to support cloud-native workflows. It does that by creating so-called Manifests which map byte ranges of the remote data to chunks of a virtual Zarr array. On access to a specific region, VirtualiZarr downloads or opens the respective byte ranges over the network and merges the data to form the actual datacubes requested. While the Manifests are locally persisted, the data is not; this requires not only a continuous online connection but also repeated downloads of the same data on subsequent accesses. For cloud-native workflows this may be sufficient, as compute often happens on the same infrastructure as the data. However, this setup puts a bandwidth burden on shared infrastructure or in some instances cannot be used at all, for example on high performance computing (HPC) clusters with segmented network architectures. Thus, repeated data downloads make local-first experimentation often challenging and need to be addressed on the data management side.

A contrasting approach is taken by xcube (Brandt & Fomferra, n.d.). xcube also transforms online data into Zarr datacubes, but explicitly duplicates the data upfront. For very large datasets, this may result in restrictions or even failure due to storage limitations, as the complete dataset needs to be converted at once.

While VirtualiZarr was explicitly built around a no-data-duplication principle, Smart-Geocubes was built as an independent library around the idea of data duplication to a fast, local storage. Meanwhile, xcube requires the conversion, thus download and storage, of the whole dataset without the ability to incrementally add data to the local store. Further, Smart-Geocubes was built with custom parallelisation through frameworks such as Ray in mind, while xcube, cubo and VirtualiZarr are strongly integrated in the Xarray-Dask ecosystem.

Software design

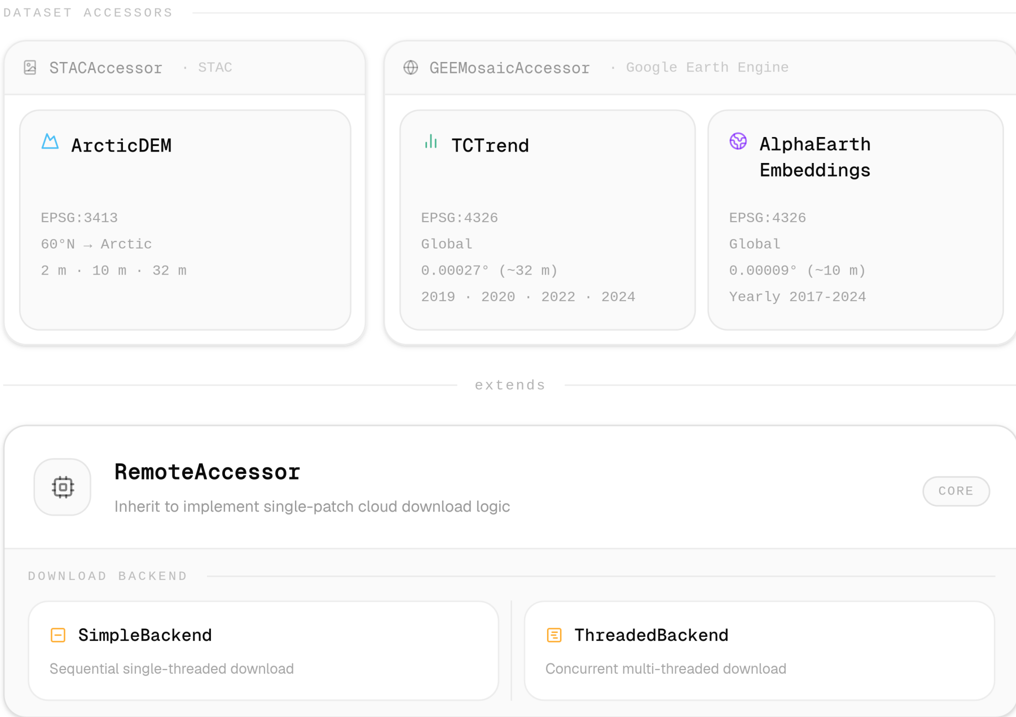

Figure 1: Architecture of the Dataset Accessors

The central design decision was adopting a procedural download paradigm: a local `Zarr` store is treated as a persistent cache incrementally populated on demand. This balances offline access to downloaded regions against avoiding upfront data transfer costs: Streaming would result in repeated downloads and thus unnecessary data transfers, while pre-downloading entire archives is impractical for terabyte-scale datasets and often not needed.

The choice of `Zarr` as the storage format stems from its read and write performance and from its inherent support for sparsity within data due to its chunk-based format (Zarr Developers, n.d.). Further, `Zarr` has started to become the standard format for geospatial data in recent years with major support and integration through libraries like `Xarray` (Hoyer & Hamman, 2017). With the development of `Icechunk`, `Zarr` was extended by a transactional storage engine allowing for safe, semi-coordinated reads and writes (Earthmover, n.d.). Thus, utilising `Icechunk` as the `Zarr` storage backend allowed `Smart-Geocubes` to ensure that parallel writes do not conflict or result in data loss, allowing safe integration into parallelisation frameworks like `Ray` (Moritz et al., 2018). Interoperability between these two libraries marks another core principle: data stored by `Smart-Geocubes` can be opened by both by `Xarray` and natively as `Zarr` via `Icechunk`.

The architecture is separated into three layers: dataset-specific accessors, remote source adapters, and the storage abstraction. The `RemoteAccessor` abstract base class forms the core, implementing common logic for patch identification, download coordination, subsetting and location maths under the hood. Source-dependent accessors inherit from this class to define remote-specific retrieval, while dataset-specific accessors further inherit from the source-dependent accessors to specify configuration and metadata. The inheritance hierarchy enables a simple way to extend the catalogue of data available through `Smart-Geocubes`: adding a new dataset from an existing source, for example a new Image Collection from Google Earth Engine, requires only a dataset-specific accessor, while supporting a new remote source requires implementing a source adapter that inherits from `RemoteAccessor`.

Two `DownloadBackend` implementations coordinate downloads and writes: The `SimpleBackend` processes patches sequentially, straightforward and easy to debug, while the `ThreadedBackend` uses worker threads and queues to download and write in parallel, providing higher throughput at the cost of added complexity. The user can select the backend based on their network setup; overall the `ThreadedBackend` proved stable and is the recommended backend due to its advanced efficiency.

As of July 2026, with an accessor for Google Earth Engine and one for any STAC catalogue, two remote accessors are implemented. Three different datasets, each with various versions, are implemented as well: ArcticDEM (2m, 10m and 32m versions) (Porter et al., 2023), Tasseled Cap Trends (2019, 2020, 2022 and 2024 versions) (Nitze et al., 2024), and AlphaEarth Satellite Embeddings (Brown et al., 2025). Further, visualisation helpers are built in to quickly get an overview of the current downloaded state. Future versions may get support for 4D data (e.g. elevation change), UTM zone handling, more remote sources and datasets, such as Sentinel-1 and 2.

Research impact statement

`Smart-Geocubes` has demonstrated significant research impact, first as a core component of our `darts-nextgen` data pipeline, a pan-Arctic-scale deep learning segmentation pipeline built on 3m PlanetScope imagery, 10m Sentinel-2 imagery, 2m ArcticDEM elevation data, and 30m Landsat Tasseled Cap Trend data to detect and map retrogressive thaw slumps, a rapid mass wasting process across the Arctic permafrost region (Hölzer et al., 2024; Nitze et al., 2025). Further, `Smart-Geocubes` became an essential data-engineering component in at least one other paper, where it was utilised to download and persist the ArcticDEM dataset at 2m resolution (Nesterova et al., 2026).

AI usage disclosure

LLMs and adjacent tools were used across various aspects of this library: VSCode's integrated GitHub Copilot and Claude Code were used to autocomplete code, to write some tests, to generate parts of the documentation, and to create the changelog. All outputs were manually reviewed and, when possible, tested by the authors. As the work was conducted over several years, different models were utilised. Only models available in the free student tier of GitHub Copilot and in the paid Claude Pro plan were used.

The MiniMax-M2.7 model hosted by Blablador in the Jülich Supercomputing Centre was used via the Jan desktop app to correct the spelling and grammar of this paper. Further, Anthropic Claude Agent was used to create a mock-review of this paper and repository to highlight potential improvements.

Acknowledgements

Most of this library was written as part of the Permafrost Discovery Gateway project, funded by the Google.org Impact Challenge on Climate Change. Additional support was provided by the PeTCaT project funded by Schmidt Sciences.

References

Brandt, G., & Fomferra, N. (n.d.). *xcube – Python package for Earth observation data cubes*. Zenodo. https://doi.org/10.5281/zenodo.5675969

Brown, C. F., Kazmierski, M. R., Pasquarella, V. J., Rucklidge, W. J., Samsikova, M., Zhang, C., Shelhamer, E., Lahera, E., Wiles, O., Ilyushchenko, S., Gorelick, N., Zhang, L. L., Alj, S., Schechter, E., Askay, S., Guinan, O., Moore, R., Boukouvalas, A., & Kohli, P. (2025). *AlphaEarth foundations: An embedding field model for accurate and efficient global mapping from sparse label data*. https://doi.org/10.48550/ARXIV.2507.22291

Earthmover. (n.d.). *Icechunk*. https://github.com/earth-mover/icechunk

Google LLC. (n.d.). *Xee: An Xarray extension for Google Earth Engine*. https://github.com/google/Xee

Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). Google earth engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, *202*, 18–27. https://doi.org/10.1016/j.rse.2017.06.031

Hölzer, T., Küpper, J., Nitze, I., & Heidler, K. (2024). *DARTS-nextgen*. https://github.com/awi-response/darts-nextgen

Hoyer, S., & Hamman, J. (2017). Xarray: N-D labeled arrays and datasets in python. *Journal of Open Research Software*, *5*(1). https://doi.org/10.5334/jors.148

Joseph, G. (n.d.). *stackstac: Turn a STAC catalog into a Dask-based xarray*. https://github.com/gjoseph92/stackstac

Montero, D., Aybar, C., Ji, C., Kraemer, G., Söchting, M., Teber, K., & Mahecha, M. D. (2024). On-demand earth system data cubes. *IGARSS 2024 - 2024 IEEE International Geoscience and Remote Sensing Symposium*, 7529–7532. https://doi.org/10.1109/IGARSS53475.2024.10640742

Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., & Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. https://doi.org/10.48550/arXiv.1712.05889

National Oceanography Centre. (2026). *NOC leads £7 million project to put petabytes of environmental data in everyone's pocket*. National Oceanography Centre news. https://www.noc.ac.uk/news/noc-leads-ps7-million-project-put-petabytes-environmental-data-everyones-pocket

Nesterova, N., Leibman, M., Stadie, C., Hölzer, T., Nitze, I., Tarasevich, I., Maier, K., Vasileva, M., Lantuit, H., Küpper, J., & Grosse, G. (2026). Rapid increase in West Siberia's retrogressive thaw slumps since 1964 associated with Arctic winter warming. *Scientific Reports*, *16*(1), 18164. https://doi.org/10.1038/s41598-026-56146-9

Nicholas, T., Hagen, N., Harkins, S., Barciauskas, A., Jones, M., Signell, J., Nag, A., Hidalgo, G., Augspurger, T., & Abernathey, R. (2024). *VirtualiZarr*. https://github.com/zarr-developers/VirtualiZarr

Nitze, I., Heidler, K., Nesterova, N., Küpper, J., Schütt, E., Hölzer, T., Barth, S., Lara, M. J., Liljedahl, A. K., & Grosse, G. (2025). DARTS: Multi-year database of AI-detected retrogressive thaw slumps in the circum-arctic permafrost region. *Scientific Data*, *12*(1), 1512. https://doi.org/10.1038/s41597-025-05810-2

Nitze, I., Lübker, T., & Grosse, G. (2024). *Pan-Arctic Visualization of Landscape Change (2003-2022), Arctic PASSION Permafrost Service* [Dataset]. PANGAEA. https://doi.org/10.1594/PANGAEA.964814

Open Data Cube. (n.d.). *odc-stac: Load STAC items into Xarray datasets*. https://github.com/opendatacube/odc-stac

Porter, C., Howat, I., Noh, M.-J., Husby, E., Khuvis, S., Danish, E., Tomko, K., Gardiner, J., Negrete, A., Yadav, B., Klassen, J., Kelleher, C., Cloutier, M., Bakker, J., Enos, J., Arnold, G., Bauer, G., & Morin, P. (2023). *ArcticDEM - mosaics, version 4.1*. Harvard Dataverse. https://doi.org/10.7910/DVN/3VDC4W

Zarr Developers. (n.d.). *Zarr-Python*. https://github.com/zarr-developers/zarr-python